

MOVELINK

System-Dokumentation & Traceability-Report

Anforderungs- & Abdeckungsanalyse der MoveLink App

Status: Freigegeben

Sprache: Deutsch

Autoren: Erlind & AI Assistant Antigravity

Dokumentenversion: v1.1.0 (Scraped Live)

Veröffentlichungsdatum: 20. Juni 2026

Gliederung (Inhaltsverzeichnis)

Dieses Dokument gliedert sich in die folgenden Abschnitte:

1. Anwendungsfälle (Use Cases)	3
2. Systemanforderungen (Requirements)	5
Funktionale Anforderungen //Was für Instanzen existieren denn eigentlich.	5
Nicht-funktionale Anforderungen	5
Randbedingungen	5
3. Architekturentscheidungen (ADR / Abwägungen)	7
Abwägungen	7
4. System-Architektur (C4 Modell)	9
4.1 Embedded Sensor-Firmware (Xiao MCU)	9
4.2 Mobile App (React Native)	10
4.3 Datenbank & Backend (PostgreSQL & Node.js)	11
5. Rückverfolgbarkeits-Matrix (Traceability)	12
6. Anhang: C4-Architekturdiagramme	14
6.1 System-Kontext-Diagramm (C4 Level 1)	14
6.2 Container-Diagramm (C4 Level 2)	14
6.3 Komponenten-Diagramm: Embedded Sensor-Firmware (C4 Level 3)	15
6.4 Komponenten-Diagramm: Mobile App (React Native - C4 Level 3)	15
6.5 System-Daten- & Kontrollfluss-Diagramm	16

1. Anwendungsfälle (Use Cases)

In diesem Abschnitt werden die primären Anwendungsfälle (Use Cases) des MoveLink-Systems beschrieben. Diese Use Cases definieren die Benutzerinteraktionen und Systemantworten für das Verbinden von Geräten, das Echtzeit-Training und das Einsehen historischer Daten.

Hier werden die primären Interaktionen zwischen dem Trainierenden und dem MoveLink-System beschrieben.

UC-1 – Trainingsgerät verbinden

- **Akteur:** Trainierender
- **Vorbedingung:** Trainingsgerät ist eingeschaltet und befindet sich in Reichweite.
- **Beschreibung:** Als Trainierender möchte ich mein Trainingsgerät mit der App verbinden, um Trainingsdaten erfassen zu können.
- **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
- **Ausgabe*:** Die App zeigt eine Möglichkeit/einen Reiter für Hardwaregeräte an.
- 2. **Eingabe:** Der Trainierende klicke auf den Reiter "Hardwaregeräte".
- **Ausgabe*:** Die App zeigt eine Liste der verfügbaren Hardwaregeräte sowie den aktuellen Verbindungsstatus an.
- 3. **Eingabe:** Der Trainierende wählt ein Hardwaregerät aus der Liste aus und klickt auf "Verbinden".
- **Ausgabe*:** Die App zeigt die Detailansicht des ausgewählten Geräts und bestätigt den erfolgreichen Verbindungsaufbau.

UC-2 – Echtzeit-Training überwachen

- **Akteur:** Trainierender
- **Vorbedingung:** Das Trainingsgerät ist erfolgreich mit der App verbunden.
- **Beschreibung:** Ich als Trainierender möchte mein Training in Echtzeit überwachen können, um direkt Feedback zu meiner Ausführung zu erhalten.
- **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
- **Ausgabe*:** Die App zeigt die Option zum Starten eines Trainings an.
- 2. **Eingabe:** Der Trainierende klickt auf "Training starten".
- **Ausgabe*:** Die App zeigt die Detailansicht des ausgewählten Trainings sowie die Start-Schaltfläche.
- 3. **Eingabe:** Der Trainierende drückt den "Start"-Button.
- **Ausgabe:** Die App demonstriert grafisch die auszuführende Übungsbewegung. (Bezug: **FA8**)*

4. **Eingabe:** Der Trainierende führt die Bewegung aus.

- **Ausgabe:** Die App visualisiert die Bewegung in Echtzeit, vergleicht sie mit der Zielvorgabe und gibt positives Feedback bei korrekter Ausführung. (Bezug: FA5, FA6, FA9)*

UC-3 – Trainingsdaten einsehen

- **Akteur:** Trainierender
- **Vorbedingung:** Mindestens eine aufgezeichnete Trainingseinheit ist in der Datenbank vorhanden.
- **Beschreibung:** Ich als Trainierender möchte vergangene Trainingseinheiten einsehen können, um meine Fortschritte zu verfolgen.
- **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
 - **Ausgabe*:** Die App bietet eine Option zum Einsehen des Trainingsverlaufs.
 2. **Eingabe:** Der Trainierende navigiert zum Reiter "Trainingseinheiten".
 - **Ausgabe*:** Die App listet alle vergangenen Trainingseinheiten chronologisch auf.
 3. **Eingabe:** Der Trainierende wählt eine Trainingseinheit aus der Liste aus.
 - **Ausgabe*:** Die App bereitet die historischen Bewegungsdaten grafisch und statistisch auf.

2. Systemanforderungen (Requirements)

Dieser Abschnitt enthält die funktionalen Anforderungen (FA), nicht-funktionalen Anforderungen (NF) und Randbedingungen (R) des MoveLink-Systems.

Dieses Dokument definiert die funktionalen und nicht-funktionalen Anforderungen sowie die Randbedingungen des MoveLink-Systems.

Funktionale Anforderungen //Was für Instanzen existieren denn eigentlich.

FA1 – Es wird eine Applikation angeboten, welche als Input für die Steuerung für den Benutzer dient. (UC-1, UC-2, UC-3)

FA2 – Zu dem System wird ein Trainingsgerät benötigt, welche als Input für der Bewegungen des Benutzers dient. Dieses Gerät gibt die Sensorwerte als auch die Art der Bewegung zurück. (UC-1, UC-2)

FA3 – Es wird eine Datenbank benötigt, welche die Daten des Benutzers speichert. (UC-1, UC-3)

Nicht-funktionale Anforderungen

NF1 – Latenz

Die End-to-End-Latenz von der physischen Sensorbewegung bis zur visuellen Darstellung in der App muss ≤ 100 ms sein.

NF2 – Zuverlässigkeit und Reconnect

Bei Verbindungsabbrüchen muss die App den Nutzer umgehend benachrichtigen und automatische Wiederverbindungsversuche (Reconnect) starten.

NF3 – Benutzbarkeit (Usability)

Das Bluetooth-Pairing mit dem Sensor darf maximal zwei manuelle Interaktionen erfordern.

Randbedingungen

R1 – Plattform-Kompatibilität

Die mobile Applikation muss nativ oder als hybride App auf Android-Geräten lauffähig sein.

3. Architekturentscheidungen (ADR / Abwägungen)

In diesem Abschnitt werden die zentralen Architekturentscheidungen (Architecture Decision Records) und Abwägungen bezüglich Hardware, App-Framework und Auswertungstechnologie dokumentiert.

Abwägungen

Hardware/Gerät:

vorgefertigtes Gerät (Fitbit)	Hybrid (XIAO seed NRF52840)	Eigene Lösung (PCB)
+ Klein und ausgereift	+ Kostengünstig, integrierte IMU & BLE	+ Maximale Kontrolle über Formfaktor und Sensoren
- Teuer	- Gehäuse muss selbst konstruiert/gedruckt werden	- Sehr hohe Entwicklungskosten und Time-to-Market
- Schlecht erweiterbar durch andere Sensoren	- Höherer Integrationsaufwand als fertiges Konsumentenprodukt	- Komplexes PCB-Design und Fertigungsrisiko

✓ Entscheidung: XIAO seed NRF52840

App:

Ionic (Flutter)	Hybrid (React Native)	Native IOS Swift/Android Java
+ Schnelles Prototyping und einfache Web-Technologien	+ Hohe Code-Wiederverwendbarkeit und schnelle UI-Iterationen	+ Optimale Bluetooth-Leistung und direkter API-Zugriff
- Performance-Einbußen bei 50Hz Echtzeit-Diagrammen	- BLE-Bibliotheken von Drittanbietern erfordern Wartung	- Hohe Entwicklungskosten durch zwei separate Codebases
- Komplexere native Bluetooth-Anbindung über Plugins	- Performance-Overhead durch JS-Bridge bei kontinuierlichem Datenstrom	- Längere Time-to-Market und aufwendige doppelte Pflege

✓ Entscheidung: Hybrid (React Native)

Bewegungsauswertung:

Die Bewertung

Auf dem Gerät	Komplett in der App	Hybrid
+ Per Edge Impulse einfach umzusetzende Inferenz	+ Genug Rechenleistung für komplexe Deep-Learning-Modelle	+ MCU filtert/komprimiert Daten; App übernimmt Inferenz

- Begrenzte Speicher- und Rechenkapazitäten der MCU	- Hohe BLE-Datenrate (50Hz Rohdatenstrom) erforderlich	- Höhere Systemkomplexität durch geteilte Logik
- Modell-Updates erfordern Firmware-Flashen	- Erhöhter Akkuverbrauch auf dem Mobilgerät	- Komplexeres Debugging bei Übertragungsverzögerungen

✓ **Entscheidung: Auf dem Gerät (Edge-Inferenz)**

4. System-Architektur (C4 Modell)

Die Software-Architektur von MoveLink ist nach dem C4-Modell strukturiert. Im Folgenden werden die einzelnen Container des Systems (Embedded Firmware, Mobile App und Datenbank & Backend) im Detail beschrieben. Für jeden dieser Unterpunkte folgen jeweils die spezifischen Anforderungen, das zugehörige ADR (Architekturentscheidung) und die Architektur-Abbildung.

4.1 Embedded Sensor-Firmware (Xiao MCU)

Die Sensor-Firmware läuft auf dem **XIAO nRF52840 Sense Controller**. Sie erfasst Beschleunigungs- und Rotationsdaten über den LSM6DS3-Sensor, wendet Filter an und überträgt Daten per BLE oder wertet sie per Edge-Inferenz direkt auf dem Chip aus.

Anforderungen (Embedded)

FA2.1 – Das Gerät sammelt die Dreh- und Beschleunigungsdaten des Trainierenden.

FA2.2 – Das Gerät erkennt, was für eine Bewegung ausgeführt worden ist.

FA2.3 – Das Gerät bewertet die Ausführung der Bewegung.

FA2.4 – Das Gerät gibt durch die LED den Verbindungsstatus aus.

FA2.5 – Das Gerät versendet die Daten an die App.

FA2.6 – Das Gerät ist in einem Gehäuse.

Architekturentscheidung (ADR / Abwägungen)

Entscheidung: Durchführung der Inferenz auf dem Mikrocontroller (Edge-Inferenz) zur Latenzminimierung (NF1) und Reduzierung des kontinuierlichen BLE-Datenstroms.

- **Lokale Auswertung vs. Cloud-Streaming:** Das Ausführen der Inferenz-Engine direkt auf dem Xiao-Controller minimiert die Latenz (**NF1**) und spart Bandbreite bei der Funkübertragung.
- **Energiebedarf:** OLED-Display und kontinuierliche Sensordatenerfassung verbrauchen signifikant Energie, weshalb Akkulaufzeiten durch Cooldown-Zeiten und Schlafmodi im Idle optimiert werden müssen.

Architektur-Abbildung

Das Komponenten-Diagramm für diesen Container befindet sich im Anhang des Dokuments (siehe **Kapitel 6.3**). Es zeigt die inneren Module der Sensor-Firmware sowie die Kopplung von Sensor-Loop, Edge Impulse Inferenz und dem BLE Service.

4.2 Mobile App (React Native)

Die Mobile App bildet das Benutzer-Interface (UI) des Systems. Sie verbindet sich per BLE mit der Xiao MCU, visualisiert Live-Bewegungsdaten und synchronisiert sie mit dem Backend.

Anforderungen (Mobile App)

FA1.1 – Die App bietet eine Navigationsstruktur (Reiter/Menü) für Hardwaregeräte, Trainings und vergangene Einheiten.

FA1.1.1 – Die App bietet eine Seiten-Navigation (Drawer/SideNav) auf Tablets und Web.

FA1.1.2 – Die App bietet eine untere Tableiste (Tabs) auf Mobiltelefonen.

FA1.2 – Die App ermöglicht das Scannen nach verfügbaren Bluetooth-Sensorgeräten.

FA1.3 – Die App baut eine stabile Bluetooth-Verbindung zum Sensor auf.

FA1.4 – Die App demonstriert grafisch die auszuführende Übungsausführung (z. B. via Lottie-Animation).

FA1.5 – Die App visualisiert die empfangenen Sensordaten und den Übungsfortschritt in Echtzeit.

FA1.6 – Die App zeigt historische Trainingseinheiten grafisch und statistisch an.

FA1.7 – Die App zeigt Profildetails des Benutzers an.

Architekturentscheidung (ADR / Abwägungen)

Entscheidung: Verwendung von **React Native (Hybrid-App)** zur plattformübergreifenden Entwicklung mit hoher Code-Wiederverwendbarkeit und schnellen UI-Iterationen, bei gleichzeitiger Optimierung des SVG-Diagramm-Renderings für den 50Hz Datenstrom.

Architektur-Abbildung

Das Komponenten-Diagramm für diesen Container befindet sich im Anhang des Dokuments (siehe **Kapitel 6.4**). Es veranschaulicht die Benutzeroberflächen-Module und Custom React Hooks des App-Containers.

4.3 Datenbank & Backend (PostgreSQL & Node.js)

Der Datenbank- und Backend-Container dient als zentraler Speicher und API-Gateway des Gesamtsystems. Er nimmt Profile und Trainingshistorien auf und stellt diese persistent bereit.

Anforderungen (Datenbank & Backend)

FA3.1 – Das System speichert und verifiziert Benutzerprofile und Anmeldeinformationen persistent.

FA3.2 – Das System speichert aufgezeichnete Trainingseinheiten (Übungstyp, Wiederholungen, Qualität) persistent zur historischen Auswertung.

Architekturentscheidung (ADR / Abwägungen)

Entscheidung: Verwendung einer **PostgreSQL-Datenbank** zur persistenten und relationalen Ablage von Trainingsdaten und Nutzerprofilen. Das Backend wird mit **Node.js & Express** realisiert, um hohe Parallelität zu gewährleisten und asynchrone WebSocket-Verbindungen für Live-Streaming-Daten ressourcenschonend zu verarbeiten.

Architektur-Abbildungen (Container-Modell & Datenfluss)

Die Diagramme für diesen Container befinden sich im Anhang des Dokuments. Das System-Kontext-Diagramm finden Sie in **Kapitel 6.1**, das Container-Diagramm in **Kapitel 6.2**, und den End-to-End Daten- und Kontrollfluss in **Kapitel 6.5**.

5. Rückverfolgbarkeits-Matrix (Traceability)

Die folgende Matrix veranschaulicht die Beziehungen und Verknüpfungen zwischen den Use Cases, den funktionalen Anforderungen (FA), den Systemkomponenten und den entsprechenden Klassen/Code-Dateien im System. Klicken Sie auf eine Anforderungs-ID, um zu deren Definition zu springen, oder auf eine Komponente, um deren C4-Modell-Kapitel aufzurufen.

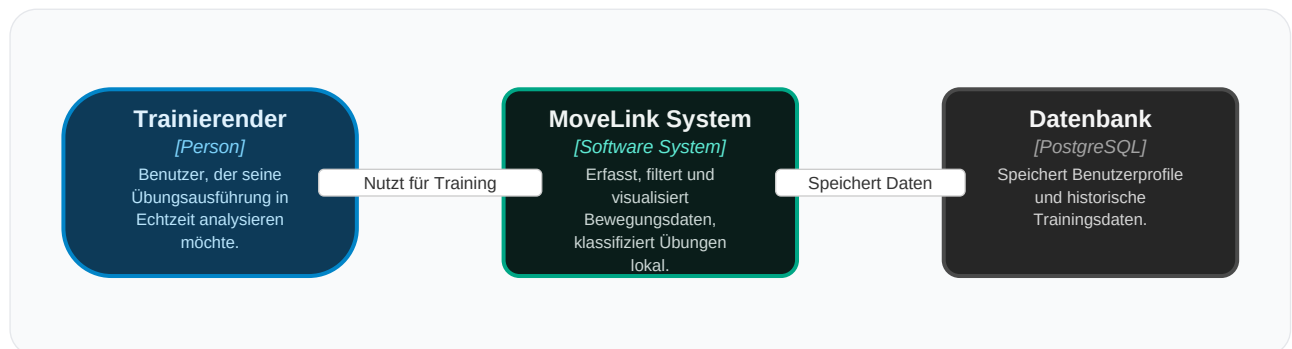
Use Case	Map	FA Ebene 1	Sub-FA	Component	Detail-FA Ebene 3	Klassenebene
UC-1 Trainingsgerät verbinden		FA1 Applikation (Mobile App)	FA1.1	SideNav	FA1.1.1 Die App bietet eine Seiten-Navigation (Drawer/SideNav) auf Tablets und Web. FA1.1.2 Die App bietet eine untere Tableiste (Tabs) auf Mobiltelefonen.	(Zielstruktur) (Zielstruktur)
			FA1.2	SensorCard	-	Props </> StatusLabel() </> opacity </> x </> t </> style </>
			FA1.3	SensorCard	-	ScanningDots() </> Props() </> StatusLabel() </> opacity </> x </> t </> style </>
			FA1.4	-	-	ScanningDots() </> Props() </> StatusLabel() </> opacity </> x </> t </> style </>
UC-2 Echtzeit-Training überwachen		FA2 Trainingsgerät (Sensor-Firmware)	FA1.5	LiveChart	-	Props </> HEIGHT </> buildPaths() </> allVals </> min </> max </>
			FA1.6	SessionCard	-	Props </> lineStr </> first </> duration() </> m </> offset() </>
			FA1.7	ProfileCard	-	Props </> chartW </> SessionCard() </> series </> stylepaths </> hasData </> minVal </> maxVal </>
			FA2.1	Sensordatenerfassung (Loop)	-	ei/getWeightNode() </> initIMU() </> readSensorData() </> initIMU() </>
UC-3 Trainingsdaten einsehen		FA2 Trainingsgerät (Sensor-Firmware)	FA2.2	Inferenz-Engine (Edge Impulse)	-	readSensorData() </> runModelInference() </>
			FA2.3	Inferenz-Engine (Edge Impulse)	-	runModelInference() </> runModelInference() </>
			FA2.4	LED- & Display-Controller	-	initFeedback() </> updateFeedback() </> sendJsonToPC() </> initFeedback() </>
			FA2.5	BLE-Streamer	-	initBLE() </> streamIMUData() </> initBLE() </> streamIMUData() </>
			FA2.6	-	-	(Zielstruktur)
			FA3.1	ProfileController / Auth Service	-	(Zielstruktur)
		FA3 Datenbank & Backend	FA3.2	Trainings-DB / Session Store	-	(Zielstruktur)

6. Anhang: C4-Architekturdiagramme

In diesem Anhang sind alle C4-Architekturdiagramme und Datenflusspläne des MoveLink-Systems übersichtlich dargestellt und beschrieben.

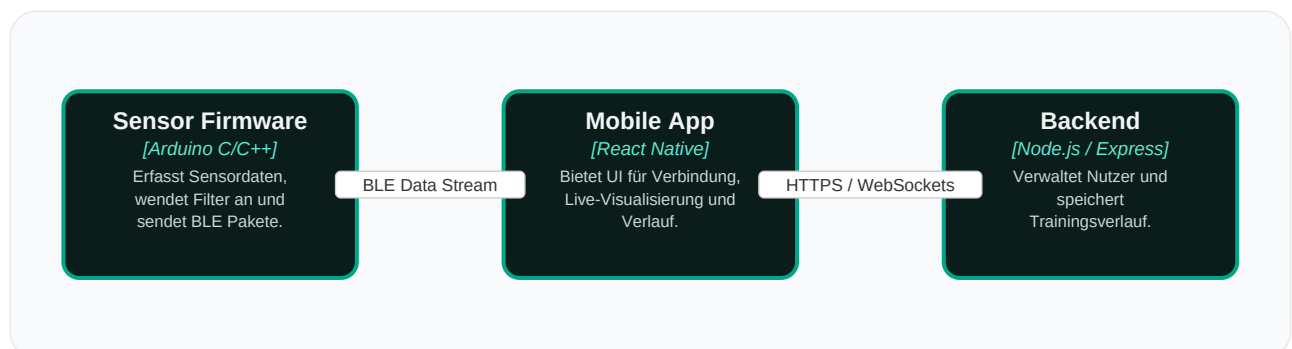
6.1 System-Kontext-Diagramm (C4 Level 1)

Das System-Kontext-Diagramm zeigt die Position des MoveLink-Systems in seiner Betriebsumgebung, die Interaktion mit dem Trainierenden sowie die Verbindung zur persistenten Datenbank.



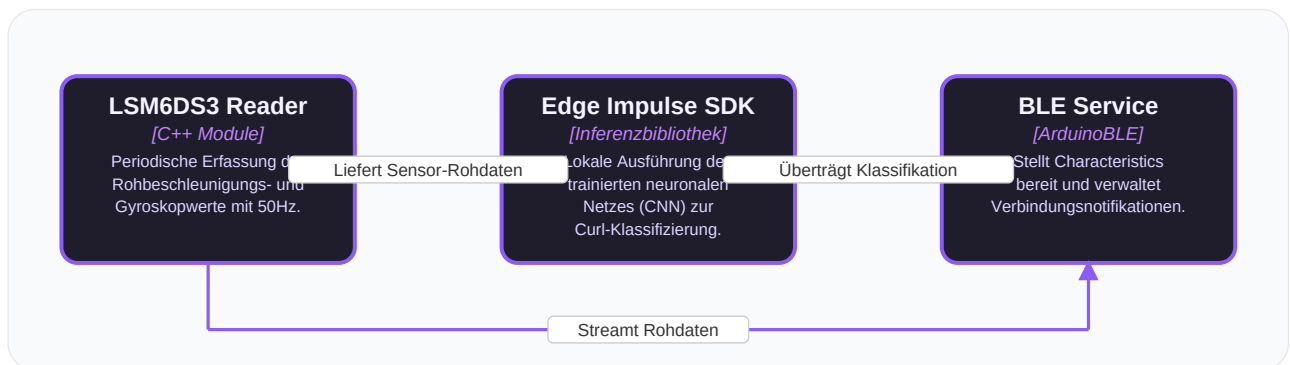
6.2 Container-Diagramm (C4 Level 2)

Das Container-Diagramm schlüsselt das MoveLink-System in seine drei eigenständigen, deploybaren Einheiten auf: Die Sensor-Firmware, die Mobile App und das Datenbank/Backend-System.



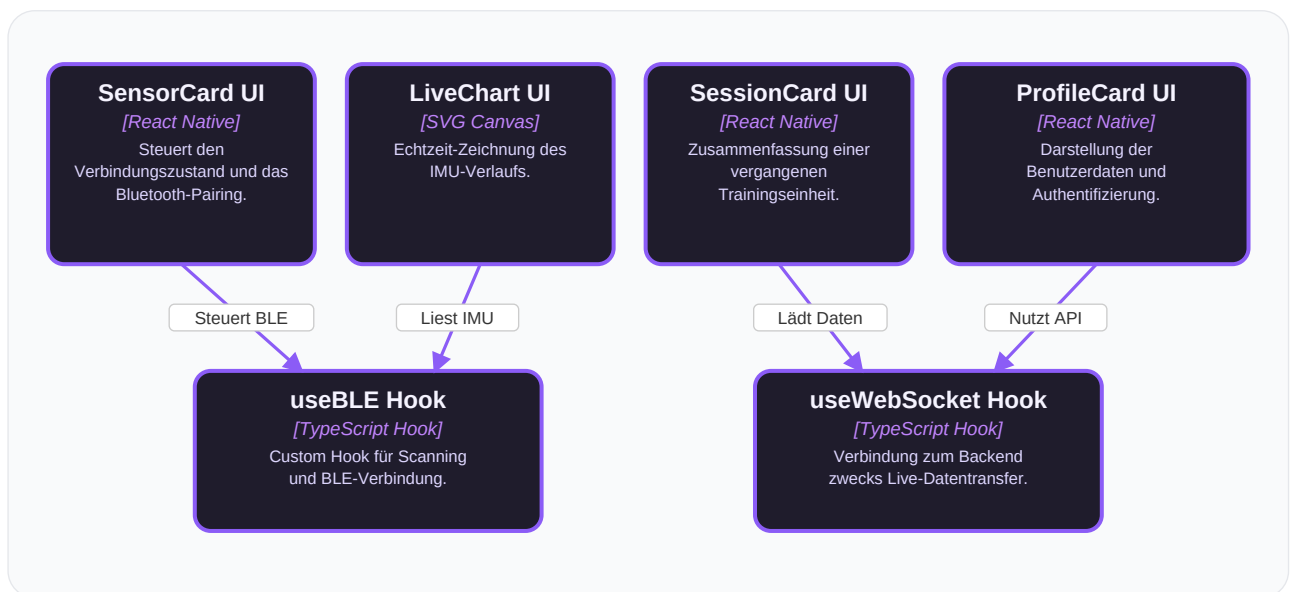
6.3 Komponenten-Diagramm: Embedded Sensor-Firmware (C4 Level 3)

Dieses Diagramm zeigt die inneren Module der Sensor-Firmware, bestehend aus Sensor-Erfassung, Edge Impulse Inferenz-Engine, LED-Controller und dem BLE-Kommunikationsmodul.



6.4 Komponenten-Diagramm: Mobile App (React Native - C4 Level 3)

Dieses Komponenten-Diagramm veranschaulicht die UI-Karten (SensorCard, LiveChart, SessionCard, ProfileCard) und deren Koppelung an die Custom Hooks useBLE und useWebSocket.



6.5 System-Daten- & Kontrollfluss-Diagramm

Visualisiert den detaillierten Hinweg des Sensor-Datenstroms (IMU -> nRF52840 -> BLE -> App -> WebSockets -> PostgreSQL) sowie den Rückweg der Steuerung und Bestätigungs-Acks im Gesamtsystem.

